SOURCE-PRIVATE TOOL-TRACE PACKETS FOR RATE-CAPPED EVIDENCE COMMUNICATION

Anonymous authors
Paper under double-blind review

ABSTRACT

Agent systems can benefit when one model uses private evidence observed by another without relaying a full trace. We formalize this as source-private evidence communication with decoder side information: a target agent receives the public task and a candidate pool, while a source agent observes private execution evidence and sends a rate-capped message. We instantiate this setting as an explicit diagnostic-code protocol benchmark where the target selects among public repair candidates and the source observes private tool-trace diagnostics. A compact explicit `REPAIR_DIAG` packet improves target-side candidate selection from 25% to 80.8–100.0% across four frozen 500-example core and held-out-family surfaces. Zero-source, shuffled-source, random same-byte, answer-only, answer-masked, target-derived, same-byte structured relay, helper/no-log, and trace-removed controls remain at target-only. Model-produced packets average 1.55–2.00 bytes, compared with roughly 366–374 bytes and 34 tokens for full hidden-log relay. A small Qwen3 model-mediated protocol-decoder sanity check gives the same qualitative direction. The result is a narrow but reproducible positive method: explicit source-private tool-trace packets can transmit the benchmark's hidden diagnostic evidence under the controls tested here and with explicit rate accounting.

1 INTRODUCTION

Multi-agent and cross-model systems increasingly divide work across components with asymmetric information. A tool-using source agent may observe execution traces, retrieval results, private memory, or environment feedback unavailable to the target agent that must make the final decision. Full trace relay is a simple interface, but it can be wasteful when the target already has useful side information such as a candidate pool or cached prior.

We study a narrower question: can a source agent send a compact, auditable message that changes the target's decision only when matched private source evidence is present? This framing follows distributed source coding and rate-distortion intuitions: a sender can communicate fewer bits when the decoder has side information, and the objective is task distortion at a communication rate rather than reconstruction of the whole source state (Slepian & Wolf, 1973; Wyner & Ziv, 1976; Shannon, 1959). Here, the target's candidate pool is decoder side information, and the source packet is evaluated by downstream selector accuracy under source controls.

We instantiate this idea in hidden-repair candidate selection. The target receives a public repair issue, buggy implementation, and four public repair candidates. The source receives a private hidden-test/tool trace containing an explicit repair diagnostic. The source model emits a compact `REPAIR_DIAG` packet, and the target-side decoder uses the packet and public candidate metadata to select a repair candidate. The packet is interpretable and byte-counted.

The contribution is intentionally scoped. We do not claim unstructured latent transfer, raw-log repair reasoning, or universal cross-model communication. We claim that a rate-capped source-private packet can carry the benchmark's hidden diagnostic evidence to a target-side candidate decoder under the controls tested here.

This makes the method closer to a syndrome-like evidence packet than to prompt compression or hidden-state transfer: the packet is only useful because the target already has candidate-side meta-

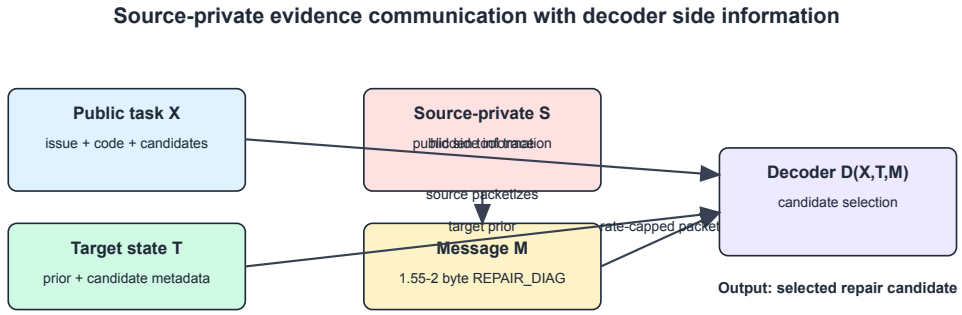

Figure 1: Source-private evidence communication with decoder side information. The source observes private evidence S and sends a rate-capped packet M. The target combines public state X, candidate state T, and M to select a candidate.

data that can decode it. The novelty is not discovering an unknown semantic representation; it is a rate-capped, source-private evidence interface with falsification controls.

2 Problem Setup

Each example has public task and candidate information X, target-side state T, source-private evidence S, a rate-capped source message M, and a decoder $D(X, T, M)$ that selects a final candidate. The target never sees S directly except in full-relay oracle baselines. A method is counted as communication only if $D(X, T, M_{\text{matched}})$ improves over no-source and source-destroying controls: zero-source, shuffled-source, random same-byte, answer-only, answer-masked, target-derived, and same-byte relay messages.

We report selector accuracy, candidate-pool recall, exact ID parity, packet validity, paired uncertainty, bytes/tokens, and source-control deltas.

3 Benchmark

The hidden-repair benchmark contains deterministic Python repair tasks, in the spirit of test-guided program-repair workflows where tests help localize and validate defects (Repair-R1). Each example has public issue text, a buggy implementation, four public candidate repairs, one gold candidate in the pool, and a source-private hidden-test/tool log. Public candidate metadata includes `handles_repair_diag=<code>`. The private source log contains `private_tool_trace: REPAIR_DIAG=<code>`.

The target prior solves exactly one quarter of examples. The main surfaces are a strict-small 160-example slice, a 500-example core slice, a 500-example held-out repair-family slice, and seed repeats over core seeds 29/31 and held-out seeds 30/32. Deterministic surfaces have candidate-pool recall 1.0, so reported improvements are selector improvements.

The synthetic design is deliberate: it gives deterministic hidden evidence, exact ID parity, complete source-destroying controls, and byte-level accounting. It is not intended to prove unconstrained program repair or real deployment generality.

4 Method

The source model receives a private hidden-test log and emits only a compact repair packet. In the main model-mediated condition, `trace_no_hint`, the private diagnostic line remains in the private log, but the copied helper line and hint are removed. Invalid packets fall back to the target prior.

Surface	Source model	Matched	Target	Control	Valid	Bytes	95% CI
core 29	Qwen3-0.6B	404/500	125/500	126/500	0.776	1.55	[0.516,0.600]
core 29	Phi-3-mini	500/500	125/500	126/500	1.000	2.00	[0.714,0.788]
core 31	Qwen3-0.6B	404/500	125/500	128/500	0.776	1.55	[0.516,0.602]
core 31	Phi-3-mini	500/500	125/500	128/500	1.000	2.00	[0.710,0.786]
held-out 30	Qwen3-0.6B	461/500	125/500	129/500	0.864	1.73	[0.632,0.712]
held-out 30	Phi-3-mini	500/500	125/500	129/500	1.000	2.00	[0.710,0.788]
held-out 32	Qwen3-0.6B	462/500	125/500	126/500	0.860	1.72	[0.634,0.716]
held-out 32	Phi-3-mini	500/500	125/500	126/500	1.000	2.00	[0.710,0.786]

Table 1: Model-mediated source-packet rows over four frozen 500-example surfaces. Control is the best source-destroying control on that surface. Intervals are paired bootstrap 95% intervals for matched minus target.

Surface	Matched	Target	Zero	Shuffled	Random	JSON	Free text	Diag-masked
core 29	1.000	0.250	0.250	0.250	0.254	0.250	0.250	0.250
held-out 30	1.000	0.250	0.250	0.250	0.254	0.250	0.250	0.250

Table 2: Deterministic threat-model rows at the 2-byte packet budget. Same-byte JSON/free-text relays and diagnostic-masked logs stay at target-only.

The target-side protocol decoder receives the public candidate pool and packet. If the packet is a valid diagnostic matching exactly one candidate's `handles_repair_diag` metadata, the decoder selects that candidate; otherwise it returns the target-prior candidate. This design makes the communication channel auditable, but it is protocol-shaped rather than a learned latent bridge. We therefore include a Qwen3 target-decoder smoke ablation in Section 9.

5 Baselines and Controls

We compare against target-only and target-wrapper/no-source baselines; zero, shuffled, random same-byte, and target-derived source-destroying controls; answer-only and answer-masked leakage controls; matched-byte hidden-log prefix, JSON, and free-text relays; full hidden-log and full diagnostic oracles; and trace-component ablations including diagnostic-masked, expected/actual-masked, test-name-masked, and trace-removed logs.

6 Results

Table 1 reports model-produced source packets across four frozen 500-example surfaces. Qwen3 reaches 80.8–92.4% accuracy and Phi-3 reaches 100%, while target-only remains 25% and source-destroying controls remain near target-only.

Removing the private trace diagnostic kills the effect: Qwen3 `raw_log_no_trace` returns to 25% with zero valid packets on all four surfaces.

7 Threat Model Results

At the 2-byte packet budget, deterministic reviewer-risk controls remain at target-only while full-log and diagnostic oracles reach 100%. Table 2 summarizes representative core and held-out surfaces.

8 Rate and Systems Analysis

The method's systems value is the far-left-rate operating point. Model-produced packets average 1.55–2.00 bytes and roughly one token. Full diagnostic text is also an oracle but costs roughly 14 bytes; full hidden-log relay reaches oracle accuracy at roughly 366–374 bytes and 34 tokens per

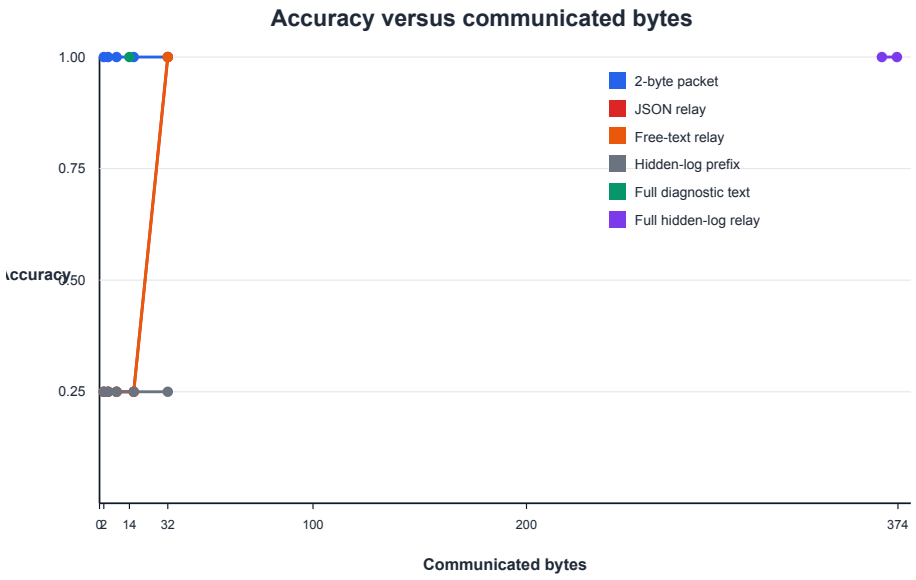

Figure 2: Accuracy versus communicated bytes, averaged over representative core and held-out reviewer-risk surfaces. This is a low-rate advantage, not dominance at all budgets: structured text relay becomes an oracle when granted enough bytes, while compact packets occupy the far-left-rate regime.

example. Structured JSON and free-text relays fail at 2 bytes but become oracles at 32 bytes because the diagnostic is then visible in parseable form.

9 Model-Mediated Protocol Decoder Smoke

To test whether the hard-coded target lookup is essential, we replace it with Qwen3-0.6B acting as a target-side selector. The model receives the target-prior fallback label, source packet, candidate labels, and candidate diagnostic metadata. On a 16-example core slice, matched packets reach 0.688 versus 0.250 target/control. On a 32-example held-out slice, matched packets reach 0.750 versus 0.250 target and 0.281 best control. This is a smoke ablation, not the main evidence or a learned latent bridge, but it reduces the hand-coded lookup concern.

10 Interpretability

The packet channel is directly auditable in this benchmark. The packet is a two-character diagnostic code; target candidate metadata states which diagnostic each candidate handles; and component-masking controls identify the private trace field that carries the signal. Failures can be audited as invalid source packets, fallback-to-prior cases, or target-decoder selection mistakes.

11 Related Work

Source coding and side information. Our setup is closest to source coding with decoder side information: Slepian-Wolf and Wyner-Ziv show that communication can be reduced when the decoder has correlated side information (Slepian & Wolf, 1973; Wyner & Ziv, 1976), and rate-distortion theory frames performance as task distortion at a communication rate (Shannon, 1959). Recent distributed indirect source-coding work is also relevant because the sender observes evidence about a task latent rather than transmitting a full reconstructable state (Tang). Semantic communication motivates task-oriented utility per transmitted bit (Xie et al., 2021).

Tool and agent communication. Tool-using agents expose observations and execution traces that may be private to one component of a system (Yao et al., 2023; Schick et al., 2023). Multi-agent frameworks motivate explicit handoff contracts between specialized agents (Wu et al., 2023). Our benchmark turns this handoff into a controlled source-private channel: the receiver sees the public task and candidate pool, while only the source sees the hidden tool-trace diagnostic.

Prompt compression and text relay. Prompt-compression methods such as LLMLingua reduce visible context length (Jiang et al., 2023). They are important budget baselines, but they do not solve the source-private setting when the target never receives the hidden evidence. We therefore compare against matched-byte text, JSON, hidden-log prefix, full diagnostic text, and full hidden-log relay rows to separate byte savings from private evidence communication.

Latent, cache, and connector communication. Direct activation and KV-cache communication methods such as C2C and KVComm communicate internal model state rather than explicit packets (C2C; KVComm). VLM connectors and JEPA-style objectives motivate future learned bottlenecks and anti-collapse diagnostics (Alayrac et al., 2022; Li et al., 2023; Assran et al., 2023; Bardes et al., 2022). The present method is deliberately narrower: it sends an auditable source-private diagnostic packet and evaluates whether matched source evidence, rather than target priors or extra prompt bytes, changes the final selection.

12 Limitations

The main limitation is scope. Candidate metadata exposes the diagnostic mapping, and the primary decoder is deterministic and protocol-shaped. The target-decoder smoke reduces this concern but remains small. The benchmark is synthetic and candidate-selection based; it does not establish unconstrained program repair, raw-log reasoning, real tool/retrieval deployment, or learned latent transfer. Structured text relay becomes an oracle when granted enough bytes. Stronger future comparisons should include learned prompt compression, C2C/KVComm-style cache transfer, and larger model-mediated target decoders under the same source-private controls.

13 Conclusion

We present a scoped positive method for source-private evidence communication. A source model reads private tool-trace diagnostics and emits a compact explicit packet. A target-side candidate decoder combines that packet with public candidate metadata to select the correct repair. The method is reproducible, interpretable, rate-accounted, and stable across the tested frozen 500-example surfaces, held-out repair families, seed repeats, source-destroying controls, structured relay baselines, and a small target-LLM decoder ablation.

References

Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: A visual language model for few-shot learning. In *Advances in Neural Information Processing Systems*, 2022.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022.

C2C. Cache-to-cache: Direct semantic communication between large language models. `https://arxiv.org/abs/2510.03215`, 2025.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Empirical Methods in Natural Language Processing*, pp. 13358–13376, 2023.

KVComm. KVComm: Enabling efficient LLM communication through selective KV sharing. https://arxiv.org/abs/2510.03346, 2025.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International Conference on Machine Learning*, 2023.

Repair-R1. Repair-R1: Better test before repair. https://arxiv.org/abs/2508.13179, 2025.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Claude E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7:142–163, 1959.

David Slepian and Jack Wolf. Noiseless coding of correlated information sources. *IEEE Transactions on Information Theory*, 19(4):471–480, 1973.

Tang. Distributed indirect source coding with decoder side information. https://arxiv.org/abs/2405.13483, 2024.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Adam White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. https://arxiv.org/abs/2308.08155, 2023.

Aaron D. Wyner and Jacob Ziv. The rate-distortion function for source coding with side information at the decoder. *IEEE Transactions on Information Theory*, 22(1):1–10, 1976.

Huiqiang Xie, Zhijin Qin, Geoffrey Ye Li, and Biing-Hwang Juang. Deep learning enabled semantic communication systems. *IEEE Transactions on Signal Processing*, 69:2663–2675, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

A Claim Boundary

We claim that explicit source-private tool-trace packets can communicate hidden diagnostic evidence to a target-side candidate decoder under rate constraints. We do not claim raw-log repair inference, learned latent transfer, universal cross-model communication, or unconstrained repair generation.

B Artifacts

Main result artifacts are tracked in the source-private tool-trace results folders. Table 3 lists the decisive manifests and scripts.

The reproduction entry points are the hidden-repair packet generator/evaluator, model packet extractor, baseline-pack builder, figure builder, and target-decoder smoke scripts; exact commands and paths are recorded in the artifact manifests.

Evidence	N / seeds	Entry point	Artifact
Model packet rows	500; 29/31/30/32	model packet extractor and baseline-pack builder	baseline-pack manifest
Reviewer controls	500; 29 and 30	hidden-repair packet evaluator	reviewer-risk manifest
Target-decoder smoke	16/32; 29 and 30	target-decoder smoke runner	target-decoder manifest
Figures and rate curve	500; 29 and 30	figure builder	figure manifest
LaTeX compile	paper source	latexmk	compile manifest

Table 3: Decisive artifact manifests. Each manifest records exact commands, paths, seeds, budgets, and summaries; the git commit provides source hashes.